

The Complete Guide to Sharding

When to split a data set across machines, how to choose the key, how consistent hashing actually works, how to reshard a live system — and everything you permanently give up the moment you do.

Earn it before you do it

The five cheaper moves first, and the signals that genuinely force a split.

The shard key is the design

Hash, range, directory, geo — cardinality, skew and whether your queries survive.

Consistent hashing, properly

The ring, virtual nodes, replica placement, and why modulo hashing is an outage.

Resharding without downtime

Dual-write, backfill, verify, flip — plus logical buckets so you never do it again.

What's inside

Read it front to back once. Part 2 (the key) and Part 6 (what you lost) are where interviews are won; Part 5 is the one candidates have never thought about.

1 Before you shard

The five cheaper moves, the three signals that actually force a split, and what sharding permanently costs you.

2 Choosing the shard key

Hash, range, directory and geo partitioning; cardinality, skew, query alignment, tenants and the celebrity problem.

3 Consistent hashing

The ring, virtual nodes, replica placement, rebalancing maths, and the alternatives worth naming.

4 Routing and topology

Client-side vs proxy vs coordinator, slot maps, redirects, the metadata service, and connection fan-out.

5 Resharding a live system

Split and merge, dual-write, backfill, verification, cutover — and the logical-bucket trick that makes it routine.

6 Living with shards

Cross-shard queries and transactions, secondary indexes, uniqueness, aggregation, hot shards, noisy tenants, backups.

7 The interview itself

How to answer a sharding question, 14 asked-in-real-interviews Q&A, and the red flags to avoid.

★ One-page cheat sheet

The night-before page: the decision table, the numbers, sharding in eight facts.

Part of the HLD concept series. Replication and quorums, caching, and the messaging layer are separate volumes; this one is only about splitting the data.

How to use this guide

Understand

Every section opens in plain English before any jargon. If a sentence needs a diagram, there is one.

Say out loud

The grey “interview line” boxes are the exact sentences that earn the point.

Defend

Every choice lists its cost. Interviewers score the trade-off, not the choice.

PARTITIONING VS SHARDING — THE WORDS

Partitioning splits one table inside one database: the engine prunes partitions per query, retention becomes a **DROP**, and you still have one write primary. **Sharding** splits the data set across independent databases with routing above them: you gain write capacity and you lose cross-shard joins, transactions and easy uniqueness. Interviewers use the words loosely; you should not.

THE ONE THING TO INTERNALISE

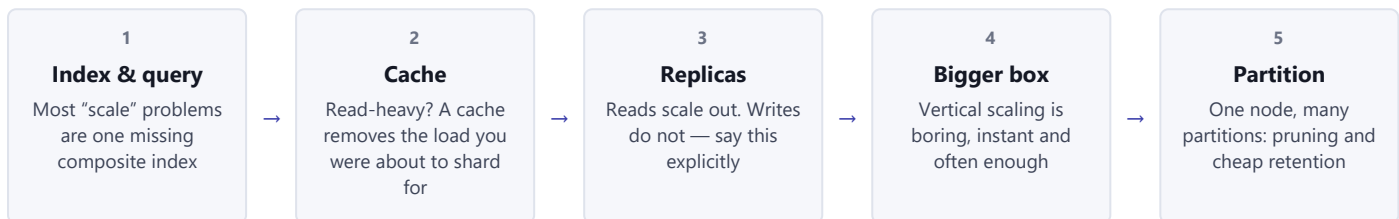
Sharding buys exactly one thing — **capacity beyond one machine** — and it charges you in every query that does not name the shard key. So the design work is not the split; it is choosing a key that keeps almost every request on one shard, and knowing what to do about the requests that cannot.

Before you shard

PART 1

Reaching for shards early is the most common way to fail a system design interview while sounding advanced. The senior move is to name the cheaper options first, then the number that rules them out.

1.1 The ladder, in order



Only when all five are exhausted does the sixth step — splitting across machines — earn its complexity. Saying that ladder out loud takes fifteen seconds and immediately separates you from the candidate whose first move is "I'll shard by user id".

1.2 The three signals that actually force a shard

Write throughput

One primary tops out around 5–10k writes/s. If the workload needs 50k, no cache or replica helps — writes must be split.

Working set vs RAM

When the hot data no longer fits in memory, every read becomes disk I/O and p99 falls off a cliff.

Operational size

A 20 TB table is not just slow: backups take a day, restores take longer, and schema changes stop being feasible.

Blast radius / residency

One tenant must not be able to take down the rest, and EU data may have to stay in the EU. Both are sharding arguments that have nothing to do with load.

INTERVIEW LINE

"Before I shard I would check whether one primary can carry this: the estimate says roughly 8,000 writes a second at peak, which is right at the edge, so yes — this one needs splitting. If it were 800 I would use a cache, read replicas and a bigger box, and I would say so rather than sharding for the sake of it."

1.3 What sharding permanently costs you

What you lose	Why	What you do instead
Cross-shard joins	The rows live in different databases; no engine can join them for you	Denormalise so the read is one shard, or join in the service layer for small result sets
Single-node transactions	ACID stops at the shard boundary	Keep the transaction inside one shard by design; across shards use a saga with compensations
Global uniqueness	A unique index is per shard, so two shards can both accept "alice@example.com"	A separate claim table or a uniqueness service, keyed so the claim lands on one shard
Cheap aggregates	COUNT(*) now means scatter-gather over every shard	Maintain counters as you write, or answer from a warehouse and say how stale it is
Simple operations	Backups, migrations, upgrades and on-call all multiply by the shard count	Automate per-shard operations from day one; never treat shard #7 as special
ORDER BY / LIMIT across shards	The global top-N needs N from every shard, merged	Fan out with a per-shard limit and merge, or precompute the ranking

Every one of those lines is an interview answer waiting to be asked. Volunteering two of them, unprompted, is what a senior candidate sounds like.

Choosing the shard key

PART 2 · THE WHOLE DESIGN

Everything else in this volume is mechanics. The shard key is the decision, and it is almost impossible to change later without a migration — which is why interviewers spend most of their time here.

2.1 Four ways to partition, and what each one does to you

Strategy	Good at	Fails at
Hash of the key	Even distribution, no hotspots, trivial routing	Range queries are gone — “orders between two dates” hits every shard
Range of the key	Range scans and time windows stay on one shard; retention is a shard drop	Monotonic keys put every new write on the last shard — the classic hotspot
Directory / lookup	Maximum flexibility: move any key anywhere, rebalance per tenant	The lookup service is now on every request path — cache it, replicate it, or it is your SPOF
Geo / tenant	Locality, data residency, and blast-radius isolation	Load is uneven by definition, and a big tenant needs its own treatment

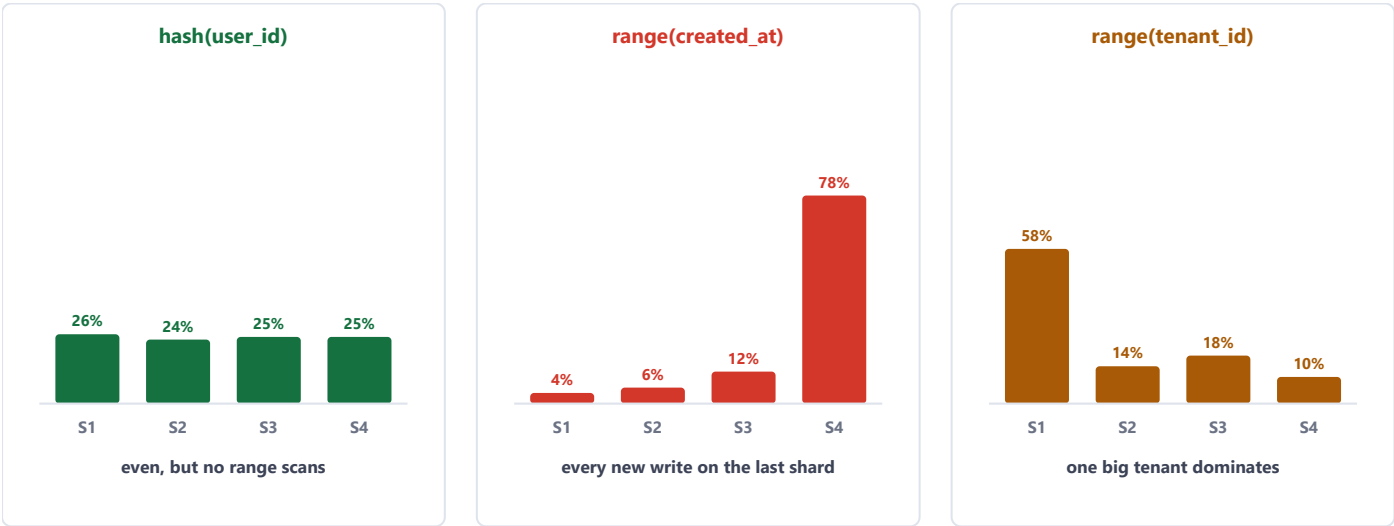


Figure 2.1 — Four shards, three keys, three futures. The distribution is a property of the key, not of the database.

2.2 The four tests a candidate key must pass

1 • High cardinality

Enough distinct values to spread over your shard count and the next one. "Country" has 200 values and half your traffic in three of them.

2 • Even distribution

No value carries a disproportionate share — and check the tail, not the mean. One key at 5% of traffic is a hot shard.

3 • Query alignment

Your top three queries must be answerable with the key in hand, or they become scatter-gather.

4 • Stability

The value must not change. Sharding on something mutable means moving rows between shards on update.

2.3 The keys that keep appearing in interviews

System	Key	Why it works
Chat / messaging	hash(conversation_id)	Every read is "this conversation's recent messages" — one shard, ordered
Social feed	hash(user_id)	The timeline belongs to one user; fan-out writes go to many shards, reads to one
E-commerce orders	hash(customer_id)	A customer's orders and their history stay together; reporting goes to the warehouse
Metrics / logs	range(time) + hash(series)	Composite: recent data on one set of shards for retention, series hashing for spread
Multi-tenant SaaS	tenant_id (directory)	Isolation, residency and per-tenant migration; big tenants get dedicated shards
Ride-hailing / maps	geohash / S2 cell	Proximity queries need spatial locality, so the key is the geography

2.4 The hot shard, and the four honest fixes

One key, one shard. That is the whole mechanism behind every celebrity problem: adding shards cannot help, because the traffic is not spread across keys — it is one key. Say the mechanism first, then the fixes in order of how much they cost you.

Reads: cache in front

A short-TTL in-process cache absorbs a viral key: 200 servers make 200 requests per TTL instead of a million per second. Cheapest fix, and it changes nothing about the data model.

Writes: salt the key

Write to **key#0 ... key#15** at random and have a worker fold them into a total. The write spreads over 16 shards; reads either read the total or sum 16 values.

Give it its own shard

For a known-large tenant, dedicate a shard (or a whole cluster). The directory strategy exists precisely so you can do this without changing anything else.

Composite key

Add a second dimension to the key — **(video_id, hour)** or **(sensor_id, day)** — so a hot entity is spread across many partitions. Cheap, but it changes every query you have already written.

THE SENTENCE THAT SCORES

“Adding shards does not fix a hot key, because a single key hashes to a single shard. I would absorb the reads in a local cache with a few seconds of TTL, and if the writes are hot I would split the key into sixteen sub-keys and fold them together asynchronously — and I would make sure those sub-keys land on *different* shards, which means no shared hash tag or prefix.”

Consistent hashing

PART 3

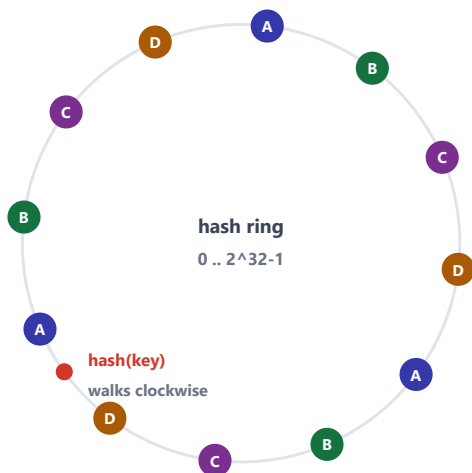
The algorithm interviewers ask you to explain by name. It answers one question: how do I add or remove a node without moving all the data?

3.1 The problem with the obvious answer

$\text{hash}(\text{key}) \% N$ is even, fast and catastrophic to change: when N goes from 4 to 5, about 80% of keys map somewhere new. For a cache that is a stampede against your database; for a data store it is a full reshuffle of every byte you own.

3.2 The ring, and virtual nodes

Hash both the keys *and* the nodes into the same space, arrange it as a ring, and a key belongs to the first node clockwise from it. Adding a node claims one arc, so only the keys in that arc move — roughly $1/N$ of them. Virtual nodes (each physical node appearing at many points) fix two remaining problems: uneven arc sizes, and the fact that without them a new node only relieves its single clockwise neighbour.



Why virtual nodes

Each physical node owns many small arcs, not one big one.
Adding a node steals a slice from every other node, so it moves $\sim 1/N$ of the keys instead of rehashing everything.
Replicas = the next R distinct physical nodes clockwise.

modulo hashing: $\text{hash}(\text{key}) \% N$

N changes and almost every key moves.

That is the outage nobody plans for.

Alternatives worth naming

Rendezvous (HRW) hashing: no ring, same minimal movement, and weights are easy to express.

Jump consistent hash: tiny and fast, but nodes can only be added or removed at the end.

Figure 3.1 — One ring, four nodes, three virtual nodes each. Replication is “the next R distinct physical nodes clockwise”, which is also how Dynamo-style stores place replicas.

3.3 The numbers to quote

keys moved when adding one node = about $1 / (N + 1)$

Going from 9 to 10 nodes moves about 10% of the keys, not 90%. With 100–256 virtual nodes per physical node, the load spread stays within a few percent of even — that is the number to say when asked “how many vnodes?”.

3.4 Where you have already met it

Redis Cluster

16,384 fixed hash slots rather than a ring — the same idea with integer arithmetic, and rebalancing moves whole slots.

Cassandra / Scylla

A token ring with 256 vnodes per node by default; replicas are the next nodes clockwise, skipping racks.

Memcached clients

Ketama, the original client-side consistent hashing implementation — the reason a node failure is not a total cache flush.

Load balancers / CDNs

Consistent hashing on a request key to keep a user or an object pinned to one backend, so its cache stays warm.

3.5 The two alternatives worth naming

- **Rendezvous (highest random weight) hashing:** for each key, compute **hash(key, node)** for every node and take the maximum. No ring, no vnodes, same minimal movement, and weights are easy. $O(N)$ per lookup, which is fine for tens of nodes.
- **Jump consistent hash:** a handful of lines, no memory, perfectly balanced — but buckets are numbered $0 \dots N-1$, so you can only add or remove at the end. Excellent for stateless sharding of work, wrong when specific nodes must be removed.
- **Fixed logical buckets:** not an algorithm so much as a design — hash into 1,024 buckets forever and map buckets to nodes in a table you can edit. Most production systems end up here, because it makes migration a routing change (Part 5).

Routing and topology

PART 4

Something has to turn a key into a connection. Where that logic lives is a real architectural choice, and each option has a failure mode interviewers like to probe.

4.1 Three places to put the routing

Where	How it works	Cost / failure mode
In the client	The library holds the topology and connects straight to the right shard — Cassandra and Redis Cluster drivers work this way	One extra hop saved, but every service embeds the map: rolling out a topology change means redeploying clients, and every client needs connections to every shard
In a proxy	A stateless layer (Vitess, ProxySQL, Twemproxy, Envoy) parses or hashes and forwards	One deploy point and connection pooling for free; adds a hop of latency and a tier to scale and page for
In a coordinator	Any node accepts any request and forwards internally — Dynamo-style	Simplest client in the world; the internal hop is on every request and a slow node slows requests it does not own

4.2 The metadata problem

Whoever routes needs the current map: which shard owns which range, and where that shard lives. That map is small, read constantly and must survive failover — so it lives in a consensus store (ZooKeeper, etcd, Consul), is cached everywhere, and carries a version number. When a client is stale it must find out cheaply: Redis Cluster answers with **MOVED** (permanent, update your map) or **ASK** (temporary, this key only), and every well-designed router has an equivalent.

4.3 The queries you must design for

Single-shard (the target)

The key is in the request, so one lookup answers it. This is the shape you denormalise *towards*. Latency is one hop and capacity scales linearly.

Scatter-gather (the tax)

No key, so every shard is asked and the results merged. Latency becomes the *slowest* shard — a p99 on one node becomes the p50 of the whole query at enough fan-out.

Three ways to make scatter-gather survivable, all worth naming: bound the fan-out (never query 200 shards for one page of results); set a per-shard deadline and return partial results marked as partial; and cache the merged answer, because the query that needed the fan-out is usually not personalised.

4.4 Connections, the thing that actually breaks

A fleet of 200 services $\times$ 20 pods $\times$ a pool of 10 connections $\times$ 32 shards is 1.28 million connections. That number is why proxies exist. Say it: **“client-side routing means every client holds a pool to every shard, so connection count grows with fleet $\times$ shards — past a certain size you need a proxy tier or transaction-mode pooling.”**

4.5 Secondary indexes across shards

Local index

Each shard indexes its own rows. Cheap and consistent — but a query without the shard key must ask every shard.

Global index

A separate structure sharded by the index key. One lookup, but it is a second write on every change and it is eventually consistent.

The honest framing

“Local index, scatter-gather read” or “global index, asynchronous write” — those are the only two options and I would pick per query.

What DynamoDB calls them

LSI is local (same partition key, extra sort key); GSI is global, with its own partitioning and its own capacity.

Resharding a live system

PART 5 · THE QUESTION NOBODY PREPARES

“You picked user_id and now one tenant is 40% of the data. Reshard it, with the site up.” This is the follow-up that separates people who have done it from people who have read about it.

5.1 The four mechanisms

Split a range
Range-partitioned stores (HBase, Bigtable, CockroachDB, MongoDB) split a hot range in two and move half. Automatic in some, a command in others.

Move logical buckets
If keys map to 1,024 buckets and buckets map to nodes, you move buckets. This is the design that makes resharding boring.

Add vnodes / slots
Consistent-hashing clusters stream the arcs or slots the new node claims. Redis Cluster and Cassandra both do this online.

Copy and cut over
The general answer when the store gives you nothing: dual-write, backfill, verify, flip — described below.

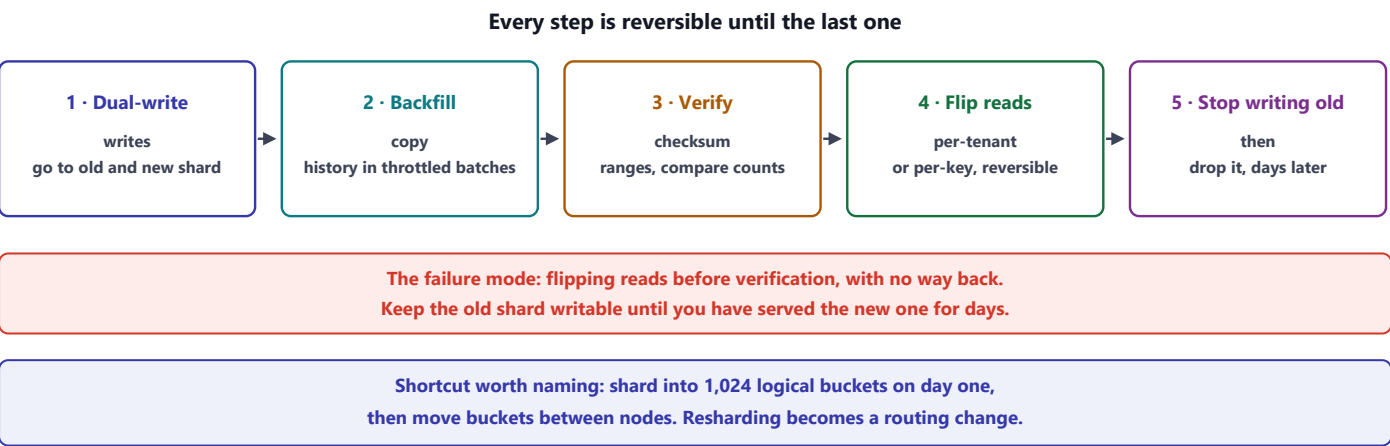


Figure 5.1 — The migration order. The only irreversible step is the last one, and it happens days after the flip.

5.2 Saying the migration out loud

The interviewer is listening for reversibility and verification, not for the copy itself. Five sentences:

- **Dual-write** every mutation to the old and the new shard, behind a flag, from the same code path — so there is one place to turn it off.
- **Backfill** history in batches, throttled and resumable, keyed by primary key ranges so a restart continues rather than starts over.
- **Verify** before trusting: row counts per range, checksums over key ranges, and a shadow-read comparison that logs mismatches without serving them.
- **Flip reads** gradually — per tenant, per key range, or by percentage — with an instant rollback, and watch error rate and latency per shard, not globally.
- **Stop writing the old shard days later**, and only drop it when you would no longer want it back.

THE TRAP TO NAME FIRST

Dual-writing from application code is *not* atomic: the old write can succeed and the new one fail, and now the copies disagree in a way no retry will notice. Either drive the copy from the database's own log (CDC) or from an outbox table written in the same transaction — and reconcile with the checksum job either way. Volunteering this is a strong senior signal.

5.3 Choosing the shard count

Pick a number you can divide, not a number you will outgrow: powers of two make splitting a shard into two clean, and hashing into a large fixed bucket space (1,024 or 4,096) decouples “how many buckets” from “how many machines” forever. The cost is a bucket-to-node table you must keep authoritative — a small price for turning every future reshard into a routing change.

Living with shards

PART 6

A sharded system is not a bigger database; it is a distributed system with different rules. These are the six places that surprise teams after the migration is over.

6.1 Transactions that span shards

There is no single-node ACID across shards, so you have three honest options: **keep the transaction inside one shard** by choosing the key so related rows co-locate (best answer, and it is a modelling decision); a **saga** — a sequence of local transactions with compensating actions and an idempotency key on every step; or **two-phase commit**, which is correct, blocking, and slow enough that most systems refuse it. Distributed SQL engines (Spanner, CockroachDB, TiDB, Vitess with 2PC) will do it for you, and the price is latency and coordination.

6.2 Uniqueness and IDs

Global uniqueness

A unique index is per shard. Use a dedicated claim table sharded *by the unique value*, so "claim this email" lands on exactly one shard and is atomic there.

ID generation

No auto-increment. Snowflake, UUIDv7 or a KGS (key-generation service handing out blocks) — ordered, compact, coordination-free.

Never shard on a monotonic key

Time or auto-increment sends every insert to the newest shard. Hash it, or prefix it with something high-cardinality.

Carry the shard hint

Encoding the shard or bucket in the public id (or the URL) means routing needs no lookup at all — a trick worth mentioning.

6.3 Aggregates, reports and the warehouse

The first "how many users signed up today?" after sharding is a scatter-gather over every shard, and the second one is a nightly job. Maintain counters as you write for the numbers users see, stream changes into a columnar warehouse for the numbers analysts ask for, and say the staleness out loud ("the dashboard is five minutes behind").

6.4 Operating N of everything

Concern	What changes	What you do
Monitoring	A healthy average hides one dying shard	Every metric per shard, plus a “worst shard” panel and alerts on skew
Backups & restore	N backups, and a restore is only consistent per shard	Automate per shard; for cross-shard consistency, restore to a timestamp and replay
Schema changes	A migration is now N migrations that can partially fail	Expand–contract, driven by a tool that tracks per-shard state and can resume
Noisy tenants	One tenant’s traffic degrades everyone on its shard	Per-tenant quotas, and move the big ones to their own shard — the directory strategy earning its keep
Rebalancing	Growth is uneven, so the layout drifts	Track bytes and QPS per shard; move buckets on a schedule rather than in an incident
Capacity	“Add a node” is now a data movement, not a deploy	Keep headroom (60–70% target), because rebalancing needs spare I/O to finish

The one-liner for this whole page: **“sharding turns a database problem into a fleet problem, so everything I do to one shard has to be automated for all of them from day one.”**

The interview itself

PART 7

Sharding questions arrive as “how would you scale this?” or as a trap: a key that looks reasonable and skews horribly. Both want the same four-step answer.

7.1 The four-step answer



7.2 The five sentences that earn points

- “One primary handles roughly 5–10k writes a second; we need 40k, so this has to split.”
- “I would shard on `conversation_id`, because every read in this product is scoped to a conversation.”
- “Consistent hashing with virtual nodes, so adding a node moves about $1/N$ of the keys and not the whole keyspace.”
- “That query has no shard key, so it becomes scatter-gather — I would bound the fan-out and cache the merged result.”
- “I would hash into 1,024 logical buckets and map buckets to nodes, so the next reshard is a routing change instead of a migration.”

INTERVIEW LINE

“I want to be explicit that sharding is the expensive option: I lose cross-shard joins, single-node transactions and easy uniqueness, and I gain write capacity. I am choosing it because the write rate is above what one primary can take, and I will design the key so that the three hot queries all stay on one shard.”

7.3 Fourteen questions with model answers

Q1 When would you shard, and when would you refuse?

When one machine cannot hold the writes, the hot data no longer fits in memory, or the data set has grown past what you can back up and migrate — or when isolation and data residency demand it. I refuse while a composite index, a cache, read replicas or a bigger instance would do, because sharding costs joins, transactions, uniqueness and operational simplicity, permanently.

Q2 Explain consistent hashing as if I have not heard of it.

Hash the keys and the nodes into the same circular space. A key belongs to the first node clockwise from it. Adding a node claims one arc of the circle, so only the keys in that arc move — about $1/N$ of them — instead of the near-total reshuffle you get from $\text{hash} \% N$. Each physical node is placed at a hundred or more points (virtual nodes) so the arcs are even and a new node takes a slice from everyone, not just from its neighbour.

Q3 What is wrong with $\text{hash}(\text{key}) \% N$?

N is in the formula, so changing the node count changes almost every mapping. For a cache that means a near-total miss storm against the database at the worst possible moment; for a store it means moving nearly all the data. Consistent hashing, rendezvous hashing, or a fixed bucket space all exist to take N out of the key's identity.

Q4 How do you pick a shard key for a chat app?

`hash(conversation_id)`, because every read is “the recent messages in this conversation” and every write appends to one. That keeps reads and writes on one shard and lets the shard hold them clustered by time. Sharding by `user_id` instead would put a two-person conversation on two shards and make every read a merge.

Follow-up: a very large group chat is a hot partition — bucket it by **`(conversation_id, time window)`** so history spreads while recent messages stay together.

Q5 One tenant is 40% of the data. What now?

That is the directory strategy's reason to exist: give the whale its own shard, or its own cluster, and leave everyone else on hash-based placement. Concretely — mark the tenant in the routing table, dual-write and backfill it to the new shard, verify with checksums, flip its reads, then stop writing the old copy. Nothing about the other tenants has to change, which is exactly why I would keep a routing table rather than pure hashing for multi-tenant data.

Q6 A single celebrity key is saturating one shard. Fix it.

Name the mechanism first: one key hashes to one shard, so more shards cannot help. For reads, put a short-TTL local cache in front so each app server fetches it once per TTL. For writes, split the key — **likes:{post}:{0..15}** written at random and folded into a total by a worker — and make sure those sub-keys hash to different shards, which means no shared hash tag.

Q7 How do you run a transaction across two shards?

First I try not to: choose the key so the rows that must change together live on one shard — that is a modelling decision and it is the right answer most of the time. When it genuinely spans shards, a saga: local transactions plus compensating actions, an idempotency key per step, and a state machine that can resume. Two-phase commit is available and correct, and it blocks on the coordinator, so I would only reach for it with a distributed SQL engine doing the work.

Q8 How do you keep an email address unique across 32 shards?

A unique index only covers its own shard, so I add a claim table sharded **by the email** itself: insert the claim first, and the insert either succeeds on that one shard or fails. The user row can then live on its own shard keyed by `user_id`. The general pattern is “route the uniqueness check by the unique value”, and it applies to usernames, slugs and idempotency keys alike.

Q9 Your query has no shard key. What happens and what do you do?

It becomes scatter-gather: every shard is asked, results are merged, and the latency is the slowest shard — so tail latency becomes typical latency as fan-out grows. Options: add a global secondary index sharded by that query's key and accept it being eventually consistent; bound the fan-out and cache the merged result; or answer it from a warehouse if it is analytical rather than user-facing.

Q10 How many shards would you start with?

Enough that each shard is comfortable at projected peak with 40% headroom, in a count I can split cleanly — so a power of two. Better still, decouple the two numbers: hash into 1,024 logical buckets on day one and map buckets to however many nodes I actually run. Adding capacity then means moving buckets, which is a routing change rather than a rehash.

Q11 How do you reshard without downtime?

Dual-write to old and new behind a flag — driven by CDC or an outbox, not a best-effort double write in app code. Backfill history in throttled, resumable batches. Verify with per-range checksums and shadow reads. Flip reads gradually with instant rollback. Keep writing the old shard for days, then drop it. The point I would stress is that every step before the last is reversible.

Q12 Where should the routing logic live?

In a proxy if I have many services or many languages: one deploy point for topology changes and connection pooling for free, at the cost of a hop. In the client if I have few services and latency matters, accepting that a topology change means redeploying clients and that connection count is fleet × shards. Either way the map lives in a consensus store, is versioned, and clients can be told they are stale.

Q13 What breaks operationally after you shard?

Averages stop being informative — one dying shard hides behind 31 healthy ones, so every metric has to be per shard with an alert on skew. Backups and restores are per shard and only consistent per shard. Migrations become N migrations that can partially fail, so they need per-shard state and resumability. And capacity planning becomes data movement, which needs spare I/O to complete.

Q14 Partitioning or sharding — what is the difference, and which do you need here?

Partitioning splits one table inside one database: the engine prunes partitions, retention becomes a partition drop, and I still have one write primary and real transactions. Sharding splits across independent databases and buys write capacity at the cost of joins, transactions and uniqueness. If the pain is query time on a huge time-series table, partition. If the pain is write throughput or total size, shard.

7.4 Red flags interviewers listen for

Saying this	Says this about you
Sharding as the first scaling move	Has not priced the cheaper options
"Shard by user_id" with no access-pattern argument	Guessing, not designing
Sharding on a timestamp or auto-increment id	Has built a write hotspot on the newest shard
"Add more shards" for a hot key	Does not know that one key maps to one shard
Using hash % N in a design	Has never resized a cluster
No answer for cross-shard joins	Has not thought past the happy path
Auto-increment IDs after sharding	Collisions, or a single sequence that is now a bottleneck
"We'll just reshard later"	Has never done it; there is no "just"
Dual-writing from app code with no reconciliation	Silent divergence, and no way to detect it
Global aggregates as if nothing changed	Every COUNT(*) is now a fan-out

7.5 Real systems worth name-dropping

Amazon Dynamo (2007)

Consistent hashing with virtual nodes, replica placement clockwise, quorums and hinted handoff. The paper everything else in this volume descends from.

Vitess (YouTube → CNCF)

MySQL sharding as a proxy layer: vschema, keyspaces, resharding by splitting keyspace ranges online. Cite it as the industrial answer to "reshard MySQL live".

Redis Cluster's 16,384 slots

Fixed logical buckets instead of a ring, with MOVED/ASK redirects. The clearest production example of decoupling bucket count from node count.

Figma's Postgres sharding (2024)

A widely read account of picking shard keys per table group, logical sharding first and physical later. Good evidence you read post-mortems, not just papers.

One-page cheat sheet

The night-before page. If you remember only this, you can still hold a good conversation about splitting data.

The numbers

one primary ~ 5–10k writes/s · adding a node moves $\sim 1/(N+1)$ of keys · 100–256 vnodes per node

Target 60–70% utilisation per shard so rebalancing has room to finish, and keep the logical bucket count (1,024 or 4,096) far larger than the node count.

The decisions

Question	Default answer
Shard yet?	No — until write rate, working set or total size proves it
Which strategy?	Hash for even spread; directory when tenants differ wildly; range only for time-shaped data you will drop
Which key?	The one that appears in your top three queries and has high cardinality
How many shards?	A power of two — and hash into 1,024 logical buckets regardless
Rebalancing?	Consistent hashing with virtual nodes, or move logical buckets
Routing?	Proxy for many services; client-side for few and latency-critical
Cross-shard write?	Redesign the key first; then a saga; 2PC last
Unique constraint?	A claim table sharded by the unique value
IDs?	Snowflake or UUIDv7 — never a global auto-increment
Reshard?	Dual-write → backfill → verify → flip → drop, days apart

Sharding in eight facts

It buys one thing

Capacity past one machine. Everything else on the invoice is a cost.

The key is the design

Cardinality, distribution, query alignment, stability — in that order.

One key lives on one shard

So a hot key is never fixed by adding shards. Cache it or split it.

Monotonic keys are hotspots

Time and auto-increment send every insert to the newest shard.

hash % N is an outage

Changing N remaps almost everything. Ring, rendezvous, or fixed buckets instead.

Virtual nodes make it even

Many small arcs per node: even load, and a new node relieves everybody.

No shard key means fan-out

Scatter-gather latency is the slowest shard's latency. Bound it and cache it.

Resharding is reversible until the flip

Dual-write, backfill, verify, then flip — and keep the old copy for days.

The five failure modes

Name	Mechanism	Primary fix
Hot shard	One key or tenant takes a disproportionate share	Local cache, key salting, or a dedicated shard
Newest-shard hotspot	Monotonic shard key	Hash the key, or prefix with high-cardinality data
Fan-out tail	Query without the shard key touches every shard	Global index, bounded fan-out, cached merge
Rebalance storm	Migration competes with production I/O	Throttle, run off-peak, keep 30–40% headroom
Split brain in the map	Two routers disagree about ownership	Versioned topology in a consensus store, redirect on stale

IF YOU SAY NOTHING ELSE, SAY THIS

“Sharding is the expensive answer, so I only reach for it when one machine cannot take the writes, hold the working set, or be operated at that size. Then the whole design is the key: high cardinality, even distribution, and present in my top three queries — so almost every request stays on one shard. I would place it with consistent hashing over virtual nodes, or better, hash into a fixed set of logical buckets so future rebalancing is a routing change. And I would say out loud what I gave up: cross-shard joins, single-node transactions and cheap global uniqueness.”

Sources & further reading — DeCandia et al., *Dynamo: Amazon's Highly Available Key-value Store* (SOSP 2007); Karger et al., *Consistent Hashing and Random Trees* (1997); Thaler & Ravishankar on rendezvous hashing; Lamping & Veach, *A Fast, Minimal Memory, Consistent Hash Algorithm* (jump hash); Martin Kleppmann, *Designing Data-Intensive Applications*, chapter 6; the Redis Cluster specification; Cassandra documentation on token ranges and virtual nodes; the Vitess documentation on resharding; Figma's engineering blog on sharding Postgres; Notion's write-up on sharding its Postgres monolith; MongoDB documentation on chunk splitting and balancing.